

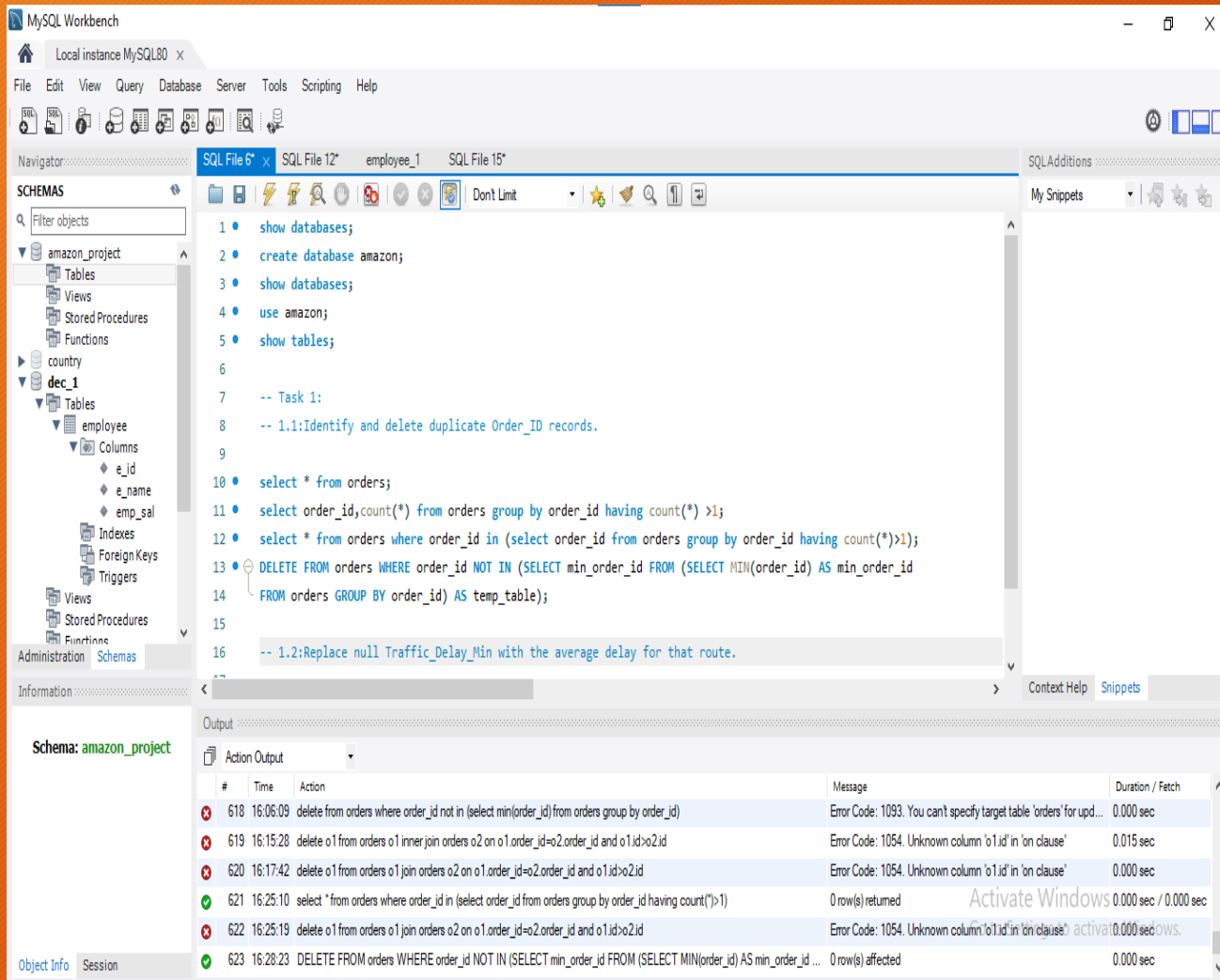
Amazon

Logistics Optimization for Delivery Routes

Data Cleaning & Preparation

TASK 1:

1. Identify and delete duplicate Order_ID records.



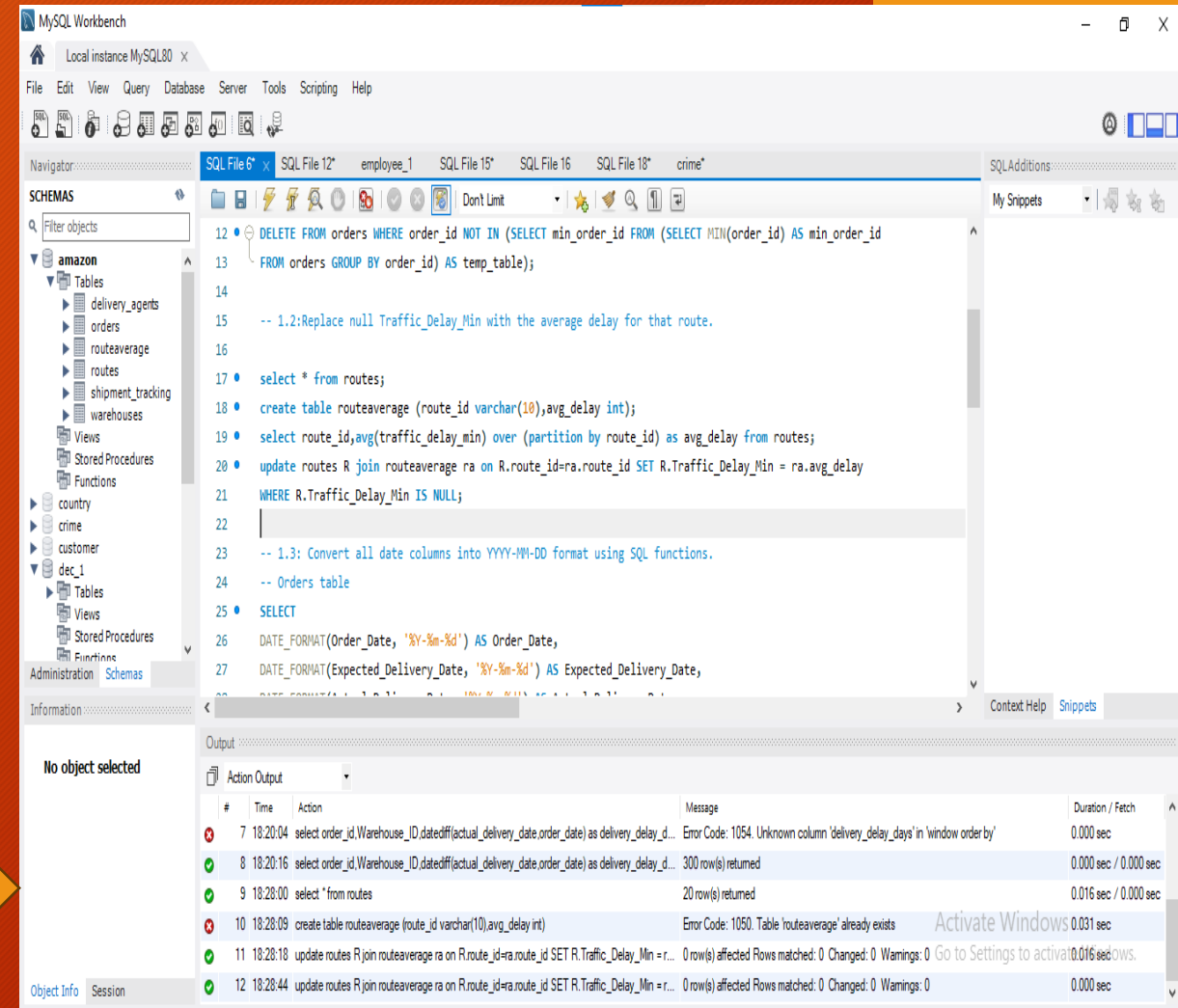
The screenshot shows the MySQL Workbench interface with the following SQL queries in the editor:

```
1 show databases;
2 create database amazon;
3 show databases;
4 use amazon;
5 show tables;
6
7 -- Task 1:
8 -- 1.1: Identify and delete duplicate Order_ID records.
9
10 select * from orders;
11 select order_id, count(*) from orders group by order_id having count(*) > 1;
12 select * from orders where order_id in (select order_id from orders group by order_id having count(*) > 1);
13 DELETE FROM orders WHERE order_id NOT IN (SELECT min_order_id FROM (SELECT MIN(order_id) AS min_order_id
14 FROM orders GROUP BY order_id) AS temp_table);
15
16 -- 1.2: Replace null Traffic_Delay_Min with the average delay for that route.
```

The Output tab shows the results of the queries:

#	Time	Action	Message	Duration / Fetch
618	16:06:09	delete from orders where order_id not in (select min_order_id from orders group by order_id)	Error Code: 1093. You can't specify target table 'orders' for update...	0.000 sec
619	16:15:28	delete o1 from orders o1 inner join orders o2 on o1.order_id=o2.order_id and o1.id>o2.id	Error Code: 1054. Unknown column 'o1.id' in 'on clause'	0.015 sec
620	16:17:42	delete o1 from orders o1 join orders o2 on o1.order_id=o2.order_id and o1.id>o2.id	Error Code: 1054. Unknown column 'o1.id' in 'on clause'	0.000 sec
621	16:25:10	select * from orders where order_id in (select order_id from orders group by order_id having count(*) > 1)	0 row(s) returned	0.000 sec / 0.000 sec
622	16:25:19	delete o1 from orders o1 join orders o2 on o1.order_id=o2.order_id and o1.id>o2.id	Error Code: 1054. Unknown column 'o1.id' in 'on clause'	0.000 sec
623	16:28:23	DELETE FROM orders WHERE order_id NOT IN (SELECT min_order_id FROM (SELECT MIN(order_id) AS min_order_id ...	0 row(s) affected	0.000 sec

2. Replace null Traffic_Delay_Min with the average delay for that route.



The screenshot shows the MySQL Workbench interface with the following SQL queries in the editor:

```
12 DELETE FROM orders WHERE order_id NOT IN (SELECT min_order_id FROM (SELECT MIN(order_id) AS min_order_id
13 FROM orders GROUP BY order_id) AS temp_table);
14
15 -- 1.2: Replace null Traffic_Delay_Min with the average delay for that route.
16
17 select * from routes;
18 create table routeaverage (route_id varchar(10), avg_delay int);
19 select route_id, avg(traffic_delay_min) over (partition by route_id) as avg_delay from routes;
20 update routes R join routeaverage ra on R.route_id=ra.route_id SET R.Traffic_Delay_Min = ra.avg_delay
21 WHERE R.Traffic_Delay_Min IS NULL;
22
23 -- 1.3: Convert all date columns into YYYY-MM-DD format using SQL functions.
24 -- Orders table
25 SELECT
26 DATE_FORMAT(Order_Date, '%Y-%m-%d') AS Order_Date,
27 DATE_FORMAT(Expected_Delivery_Date, '%Y-%m-%d') AS Expected_Delivery_Date,
```

The Output tab shows the results of the queries:

#	Time	Action	Message	Duration / Fetch
7	18:20:04	select order_id, Warehouse_ID, datediff(actual_delivery_date, order_date) as delivery_delay_d...	Error Code: 1054. Unknown column 'delivery_delay_days' in 'window order by'	0.000 sec
8	18:20:16	select order_id, Warehouse_ID, datediff(actual_delivery_date, order_date) as delivery_delay_d...	300 row(s) returned	0.000 sec / 0.000 sec
9	18:28:00	select * from routes	20 row(s) returned	0.016 sec / 0.000 sec
10	18:28:09	create table routeaverage (route_id varchar(10), avg_delay int)	Error Code: 1050. Table 'routeaverage' already exists	0.031 sec
11	18:28:18	update routes R join routeaverage ra on R.route_id=ra.route_id SET R.Traffic_Delay_Min = r...	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0 Go to Settings to activate...	0.016 sec
12	18:28:44	update routes R join routeaverage ra on R.route_id=ra.route_id SET R.Traffic_Delay_Min = r...	0 row(s) affected Rows matched: 0 Changed: 0 Warnings: 0	0.000 sec

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SCHEMAS

Filter objects

amazon

Tables

delivery_agents

orders

routeaverage

routes

shipment_tracking

warehouses

Views

Stored Procedures

Functions

country

crime

customer

dec_1

Tables

Views

Stored Procedures

Functions

Administration Schemas

No object selected

SQL File 6* x SQL File 12* employee_1 SQL File 15* SQL File 16 SQL File 18* crime*

23 -- 1.3: Convert all date columns into YYYY-MM-DD format using SQL functions.

24 -- Orders table

25 • SELECT

26 DATE_FORMAT(Order_Date, '%Y-%m-%d') AS Order_Date,

27 DATE_FORMAT(Expected_Delivery_Date, '%Y-%m-%d') AS Expected_Delivery_Date,

28 DATE_FORMAT(Actual_Delivery_Date, '%Y-%m-%d') AS Actual_Delivery_Date

29 FROM Orders;

30

31 • SELECT

32 DATE_FORMAT(Checkpoint_Time, '%Y-%m-%d') AS Checkpoint_Date

33 FROM Shipment_Tracking;

34

35 -- All date columns were already stored in YYYY-MM-DD format

36 -- Therefore, no data transformation was required

37 -- DATE_FORMAT() was used to ensure consistent output format.

Result Grid

Order_Date	Expected_Delivery_Date	Actual_Delivery_Date
2025-06-08	2025-06-11	2025-06-11
2025-06-22	2025-06-27	2025-06-29
2025-06-04	2025-06-09	2025-06-11
2025-06-28	2025-07-05	2025-07-07
2025-06-04	2025-06-09	2025-06-10
2025-07-17	2025-07-19	2025-07-19
2025-06-14	2025-06-17	2025-06-18
2025-06-23	2025-06-30	2025-07-02
2025-07-11	2025-07-16	2025-07-18
2025-06-14	2025-06-21	2025-06-21

Object Info Session

3. Convert all date columns into YYYY-MM-DD format using SQL functions.

- All date columns were already stored in YYYY-MM-DD format
- Therefore, no data transformation was required
- DATE_FORMAT() was used to ensure consistent output format.

4. Ensure that no Actual_Delivery_Date is before Order_Date (flag such records).

- We validated delivery dates by comparing Actual_Delivery_Date with Order_Date.
- Records where delivery occurred before the order date were flagged as Invalid, as they indicate data inconsistencies.

MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator: SCHEMAS

Filter objects

amazon

Tables

delivery_agents

orders

routeaverage

routes

shipment_tracking

warehouses

Views

Stored Procedures

Functions

country

crime

customer

dec_1

Tables

Views

Stored Procedures

Functions

Administration Schemas

No object selected

SQL File 6* x SQL File 12* employee_1 SQL File 15* SQL File 16 SQL File 18* crime*

39

40 -- Task 1.4: Ensure that no Actual_Delivery_Date is before Order_Date (flag such records).

41 • SELECT Order_ID, Order_Date, Actual_Delivery_Date,

42 CASE WHEN Actual_Delivery_Date < Order_Date THEN 'Invalid' ELSE 'Valid' END AS Delivery_Status FROM Orders;

43

44 -- OR

45 • SELECT *, CASE WHEN Actual_Delivery_Date >= Order_Date THEN 'Valid' ELSE 'Invalid'

46 END AS Delivery_Status FROM Orders;

Result Grid

Order_ID	Customer_ID	Warehouse_ID	Route_ID	Order_Date	Expected_Delivery_Date	Actual_Delivery_Date	Delivery_Status	Delivery_Status
O0001	C0080	W004	R005	2025-06-08	2025-06-11	2025-06-11	On Time	Valid
O0002	C0566	W009	R004	2025-06-22	2025-06-27	2025-06-29	Delayed	Valid
O0003	C0899	W008	R013	2025-06-04	2025-06-09	2025-06-11	Delayed	Valid
O0004	C0311	W001	R005	2025-06-28	2025-07-05	2025-07-07	Delayed	Valid
O0005	C0304	W006	R012	2025-06-04	2025-06-09	2025-06-10	Delayed	Valid
O0006	C0719	W004	R002	2025-07-17	2025-07-19	2025-07-19	On Time	Valid
O0007	C0652	W001	R020	2025-06-14	2025-06-17	2025-06-18	Delayed	Valid
O0008	C0111	W006	R008	2025-06-23	2025-06-30	2025-07-02	Delayed	Valid

Result 8 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 14	18:41:17	SELECT Order_ID, Order_Date, Actual_Delivery_Date, CASE WHEN Actual_Delivery_Date < Order_Date THEN 'Invalid' ELSE 'Valid' END AS Delivery_Status FROM Orders;	300 row(s) returned	0.000 sec / 0.000 sec
✓ 15	18:45:25	SELECT Order_ID, DATEDIFF(Actual_Delivery_Date, Order_Date) AS Total_Delivery_Time...	300 row(s) returned	0.000 sec / 0.000 sec
✓ 16	18:58:01	SELECT Order_ID, DATEDIFF(Actual_Delivery_Date, Order_Date) AS Total_Delivery_Time...	300 row(s) returned	0.000 sec / 0.000 sec
✓ 17	19:06:01	SELECT *, CASE WHEN Actual_Delivery_Date >= Order_Date THEN 'Valid' ELSE 'Invalid' ...	300 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

Delivery Delay Analysis

TASK 2:

1. Calculate delivery delay (in days) for each orders.

Delivery delay was calculated using the DATEDIFF() function by subtracting the order date from the actual delivery date. This helps identify delayed, on-time, and invalid deliveries.

2. Find Top 10 delayed routes based on average delay days.

Average delivery delay was calculated for each route using the DATEDIFF() function. Routes were ranked in descending order to identify the top 10 most delayed routes. Invalid records were excluded to ensure accurate analysis.

The screenshot shows the MySQL Workbench interface with a SQL query executed. The query calculates the delivery delay for each order by subtracting the order date from the actual delivery date using the DATEDIFF function. The result is categorized into 'Invalid', 'On-Time', or 'Delayed' based on the delay duration.

```
-- Task 2.1: Calculate delivery delay (in days) for each order
SELECT Order_ID,
DATEDIFF(ACTUAL_DELIVERY_DATE, ORDER_DATE) AS Total_Delivery_Time,
CASE
WHEN DATEDIFF(ACTUAL_DELIVERY_DATE, ORDER_DATE) < 0 THEN 'Invalid'
WHEN DATEDIFF(ACTUAL_DELIVERY_DATE, ORDER_DATE) = 0 THEN 'On-Time'
ELSE 'Delayed' END AS Delivery_Status
FROM Orders;
```

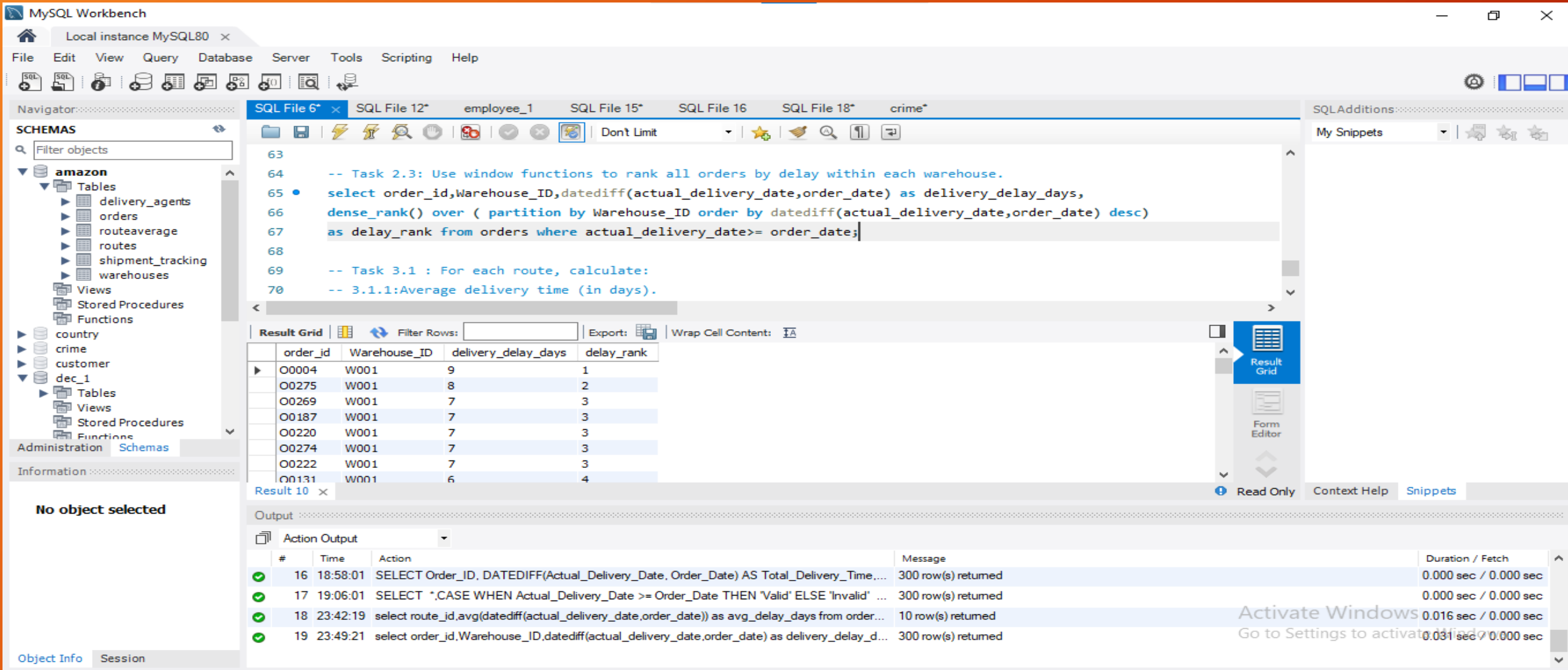
Order_ID	Total_Delivery_Time	Delivery_Status
O0001	3	Delayed
O0002	7	Delayed
O0003	7	Delayed
O0004	9	Delayed
O0005	6	Delayed
O0006	2	Delayed
O0007	4	Delayed
nnnnn	9	Delayed

The screenshot shows the MySQL Workbench interface with a SQL query executed. The query finds the top 10 delayed routes based on the average delay days. The result is ranked in descending order of average delay days.

```
-- Task 2.2: Find Top 10 delayed routes based on average delay days.
select route_id, avg(datediff(actual_delivery_date, order_date)) as avg_delay_days from orders
where actual_delivery_date >= order_date group by route_id order by avg_delay_days desc limit 10;
```

route_id	avg_delay_days
R015	6.3333
R017	6.2857
R020	5.8824
R007	5.7500
R014	5.6154
R019	5.6154
R011	5.5500
nnnnn	5.4175

3. Use window functions to rank all orders by delay within each warehouse.



The screenshot displays the MySQL Workbench interface. The SQL editor contains the following query:

```
63
64 -- Task 2.3: Use window functions to rank all orders by delay within each warehouse.
65 • select order_id, Warehouse_ID, datediff(actual_delivery_date, order_date) as delivery_delay_days,
66     dense_rank() over ( partition by Warehouse_ID order by datediff(actual_delivery_date, order_date) desc)
67     as delay_rank from orders where actual_delivery_date >= order_date;
68
69 -- Task 3.1 : For each route, calculate:
70 -- 3.1.1: Average delivery time (in days).
```

The Result Grid shows the following data:

order_id	Warehouse_ID	delivery_delay_days	delay_rank
O0004	W001	9	1
O0275	W001	8	2
O0269	W001	7	3
O0187	W001	7	3
O0220	W001	7	3
O0274	W001	7	3
O0222	W001	7	3
O0131	W001	6	4

The Output pane shows the execution of the query and other related queries:

#	Time	Action	Message	Duration / Fetch
✓ 16	18:58:01	SELECT Order_ID, DATEDIFF(Actual_Delivery_Date, Order_Date) AS Total_Delivery_Time,...	300 row(s) returned	0.000 sec / 0.000 sec
✓ 17	19:06:01	SELECT *, CASE WHEN Actual_Delivery_Date >= Order_Date THEN 'Valid' ELSE 'Invalid' ...	300 row(s) returned	0.000 sec / 0.000 sec
✓ 18	23:42:19	select route_id, avg(datediff(actual_delivery_date, order_date)) as avg_delay_days from order...	10 row(s) returned	0.016 sec / 0.000 sec
✓ 19	23:49:21	select order_id, Warehouse_ID, datediff(actual_delivery_date, order_date) as delivery_delay_d...	300 row(s) returned	0.031 sec / 0.000 sec

Orders were ranked within each warehouse based on delivery delay using window functions. This helps identify the most delayed orders per warehouse for targeted improvements.

Route Optimization Insights

Task 3:

1. For each route, calculate:
 - Average delivery time (in days).
 - Average traffic delay.
 - Distance-to-time efficiency ratio: $\text{Distance_KM} / \text{Average_Travel_Time_Min}$.

2. Identify 3 routes with the worst efficiency ratio.

MySQL Workbench interface showing a SQL query for Task 3.1. The query calculates average delivery time, average traffic delay, and distance-to-time efficiency ratio for each route. The result grid shows the following data:

Route_ID	Avg_Delivery_Time_Days	Avg_Traffic_Delay_Min	Efficiency_Ratio
R005	5.00	26.00	0.37
R004	5.10	3.00	4.28
R013	4.41	38.00	0.93
R012	4.83	29.00	0.9
R002	4.73	22.00	2.16
R020	5.88	23.00	0.55



MySQL Workbench interface showing a SQL query for Task 3.2. The query identifies the 3 routes with the worst efficiency ratio. The result grid shows the following data:

Route_ID	Efficiency_Ratio
R015	0.15
R006	0.16
R008	0.24

All route-level metrics were calculated in a single query to provide a consolidated view of performance, enabling easier comparison across routes.

3. Find routes with >20% delayed shipments.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
-- Task 3.3 :Find routes with >20% delayed shipments.
SELECT Route_ID, COUNT(*) AS Total_Orders, SUM(DATEDIFF(Actual_Delivery_Date, Order_Date) > 0) AS Delayed_Orders,
ROUND(SUM(DATEDIFF(Actual_Delivery_Date, Order_Date) > 0) * 100.0 / COUNT(*), 2) AS Delay_Percentage
FROM Orders WHERE Actual_Delivery_Date >= Order_Date
GROUP BY Route_ID HAVING Delay_Percentage > 20;
```

The Result Grid shows the following data:

Route_ID	Total_Orders	Delayed_Orders	Delay_Percentage
R005	15	15	100.00
R004	20	20	100.00
R013	17	17	100.00
R012	12	12	100.00
R002	22	22	100.00
R020	17	17	100.00
R008	16	16	100.00

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
26	08:31:10	SELECT	r.Route_ID, COUNT(o.Order_ID) AS Total_Orders, SUM(o.Delayed_Flag) ...	20 row(s) returned 0.016 sec / 0.000 sec
27	08:31:53	SELECT	Route_ID, COUNT(*) AS Total_Orders, SUM(DATEDIFF(Actual_Delivery_...	20 row(s) returned 0.032 sec / 0.000 sec
28	08:32:37	SELECT	r.Route_ID, COUNT(*) AS Total_Orders, SUM(CASE WHEN DATE...	20 row(s) returned 0.031 sec / 0.000 sec
29	08:38:15	SELECT	Route_ID, COUNT(*) AS Total_Orders, SUM(DATEDIFF(Actual_Delivery_Date, Ord...	20 row(s) returned 0.016 sec / 0.000 sec

4. Recommend potential routes for optimization.

- Optimize routes with **low efficiency ratio**
- Focus on routes with **>20% delays**
- Reduce **traffic delays** using alternate paths
- Improve **delivery scheduling (avoid peak hours)**
- Assign **better agents to high-delay routes**

Routes with high delays and low efficiency should be prioritized for optimization using better planning and traffic management.

Warehouse Performance

1. Find the top 3 warehouses with the highest average processing time.

2. Calculate total vs. delayed shipments for each warehouse.

Task 4:

The screenshot shows the MySQL Workbench interface with a query executed in the SQL editor. The query is: `SELECT Warehouse_ID, ROUND(AVG(Processing_Time_Min), 2) AS Avg_Processing_Time_Min FROM Warehouses GROUP BY Warehouse_ID ORDER BY Avg_Processing_Time_Min DESC LIMIT 3;` The result grid displays the following data:

Warehouse_ID	Avg_Processing_Time_Min
W010	171.00
W009	170.00
W007	166.00

The bottom panel shows the output log with the following entries:

#	Time	Action	Message	Duration / Fetch
30	08:52:01	SELECT w.Warehouse_ID, ROUND(AVG(w.Processing_Time), 2) AS Avg_Processing...	Error Code: 1054. Unknown column 'w.Processing_Time' in field list	0.078 sec
31	08:53:31	SELECT w.Warehouse_ID, ROUND(AVG(w.Processing_Time_Min), 2) AS Avg_Processing...	Error Code: 1054. Unknown column 'w.Warehouse_ID' in clause	0.000 sec
32	08:54:58	SELECT w.Warehouse_ID, ROUND(AVG(w.Processing_Time_Min), 2) AS Avg_Processing...	Error Code: 1054. Unknown column 'w.Warehouse_ID' in clause	0.000 sec
33	11:28:07	SELECT Warehouse_ID, ROUND(AVG(Processing_Time_Min), 2) AS Avg_Processing_Time...	3 row(s) returned	0.031 sec / 0.000 sec

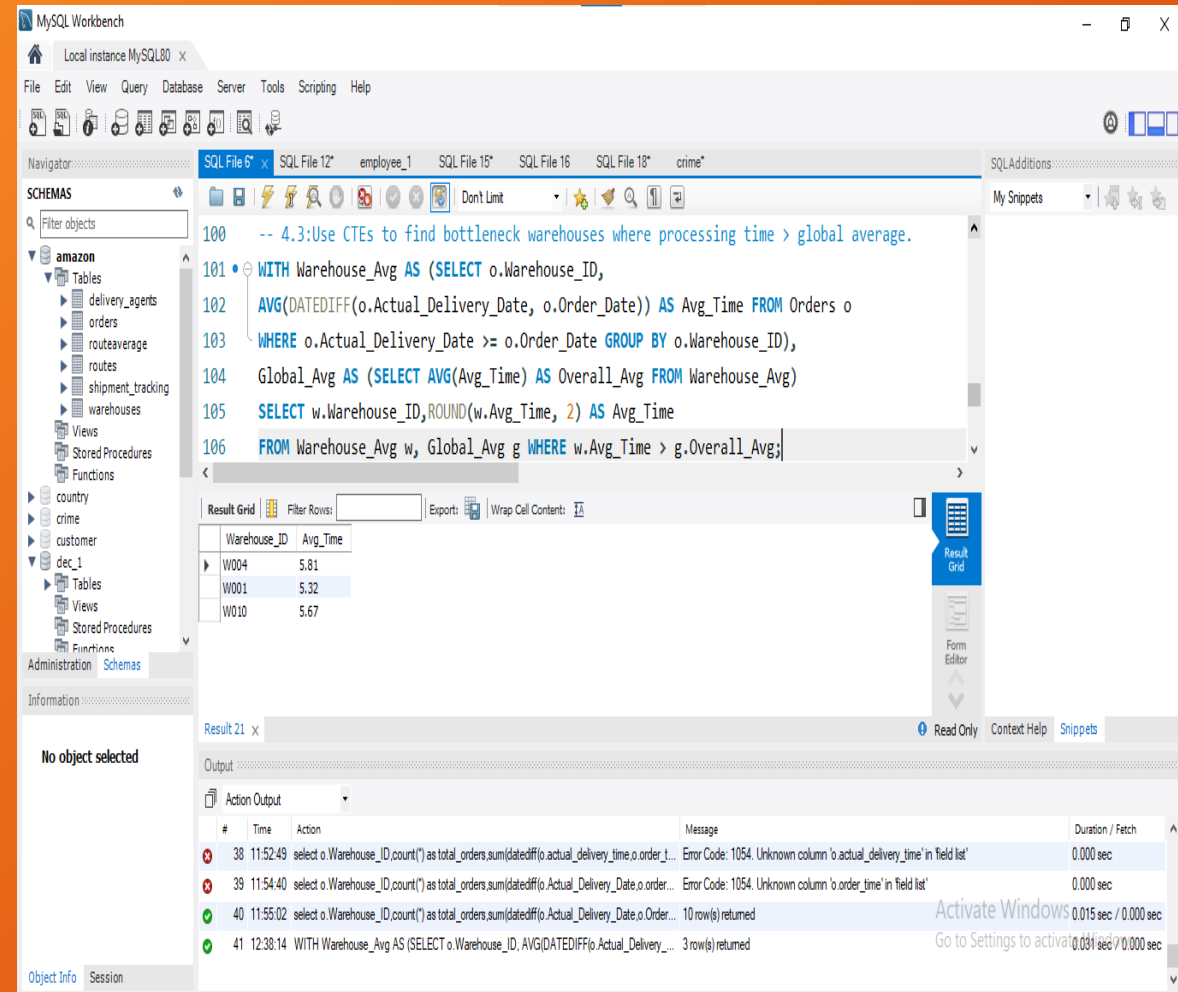
The screenshot shows the MySQL Workbench interface with a query executed in the SQL editor. The query is: `select o.Warehouse_ID, count(*) as total_orders, sum(datediff(o.Actual_Delivery_Date, o.Order_Date)>0) as delayed_orders from orders o where o.actual_delivery_date >= o.order_date group by o.warehouse_id;` The result grid displays the following data:

Warehouse_ID	total_orders	delayed_orders
W004	31	31
W009	33	33
W008	21	21
W001	34	34
W006	37	37
W010	30	30
W007	38	38

The bottom panel shows the output log with the following entries:

#	Time	Action	Message	Duration / Fetch
37	11:51:21	select o.Warehouse_ID, count(*) as total_orders, sum(datediff(o.actual_delivery_time, o.order_...	Error Code: 1054. Unknown column 'o.actual_delivery_time' in field list	0.000 sec
38	11:52:49	select o.Warehouse_ID, count(*) as total_orders, sum(datediff(o.actual_delivery_time, o.order_...	Error Code: 1054. Unknown column 'o.actual_delivery_time' in field list	0.000 sec
39	11:54:40	select o.Warehouse_ID, count(*) as total_orders, sum(datediff(o.Actual_Delivery_Date, o.order...	Error Code: 1054. Unknown column 'o.order_time' in field list	0.000 sec
40	11:55:02	select o.Warehouse_ID, count(*) as total_orders, sum(datediff(o.Actual_Delivery_Date, o.Order...	10 row(s) returned	0.015 sec / 0.000 sec

Bottleneck warehouses were identified using a CTE by comparing average delivery time per warehouse with the global average.

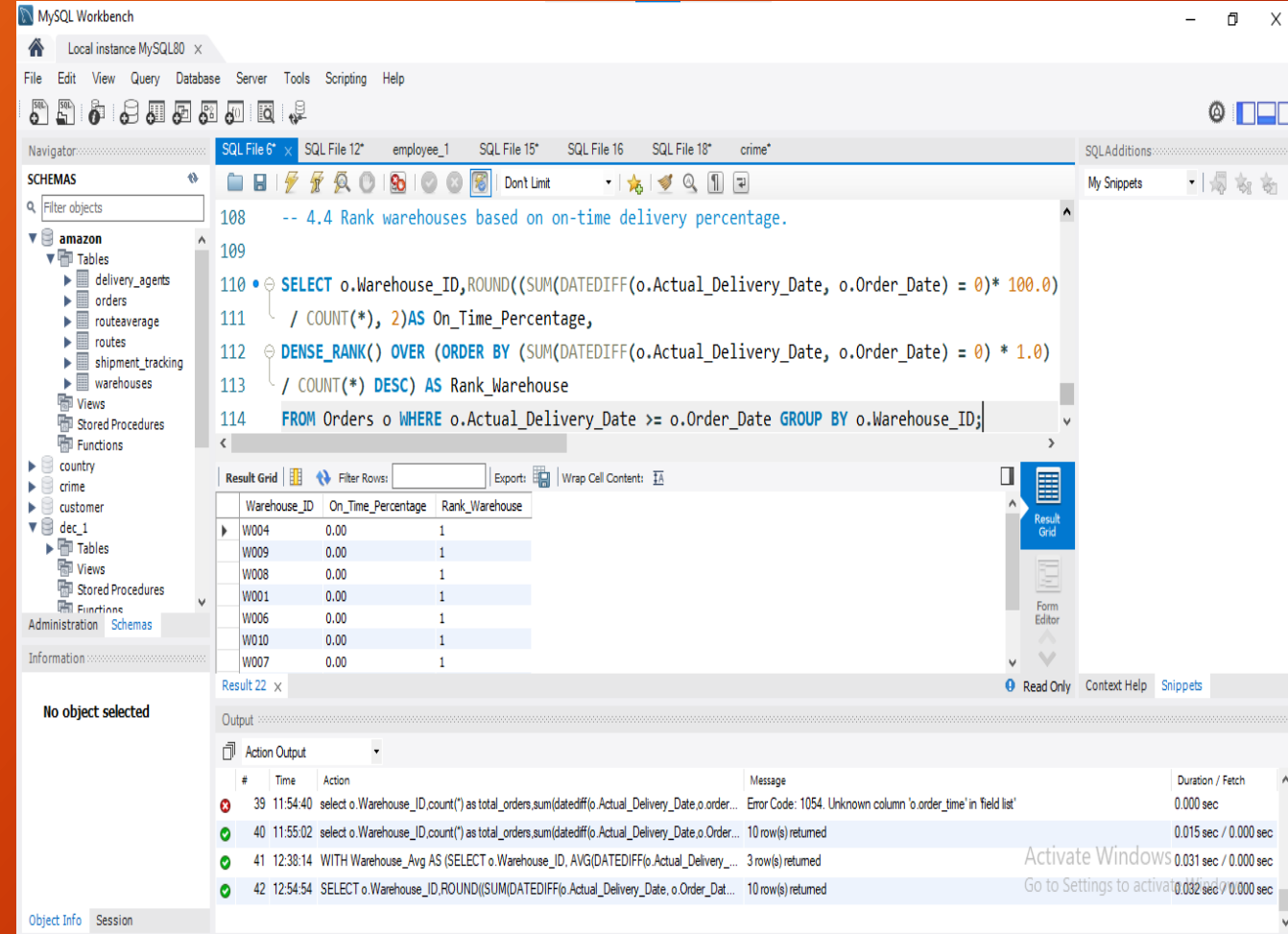


```
100 -- 4.3: Use CTEs to find bottleneck warehouses where processing time > global average.
101 WITH Warehouse_Avg AS (SELECT o.Warehouse_ID,
102     AVG(DATEDIFF(o.Actual_Delivery_Date, o.Order_Date)) AS Avg_Time FROM Orders o
103     WHERE o.Actual_Delivery_Date >= o.Order_Date GROUP BY o.Warehouse_ID),
104     Global_Avg AS (SELECT AVG(Avg_Time) AS Overall_Avg FROM Warehouse_Avg)
105     SELECT w.Warehouse_ID, ROUND(w.Avg_Time, 2) AS Avg_Time
106     FROM Warehouse_Avg w, Global_Avg g WHERE w.Avg_Time > g.Overall_Avg;
```

Warehouse_ID	Avg_Time
W004	5.81
W001	5.32
W010	5.67

3. Use CTEs to find bottleneck warehouses where processing time > global average.

4. Rank warehouses based on on-time delivery percentage.



```
108 -- 4.4 Rank warehouses based on on-time delivery percentage.
109
110 SELECT o.Warehouse_ID, ROUND((SUM(DATEDIFF(o.Actual_Delivery_Date, o.Order_Date) = 0) * 100.0)
111     / COUNT(*), 2) AS On_Time_Percentage,
112     DENSE_RANK() OVER (ORDER BY (SUM(DATEDIFF(o.Actual_Delivery_Date, o.Order_Date) = 0) * 1.0)
113     / COUNT(*) DESC) AS Rank_Warehouse
114     FROM Orders o WHERE o.Actual_Delivery_Date >= o.Order_Date GROUP BY o.Warehouse_ID;
```

Warehouse_ID	On_Time_Percentage	Rank_Warehouse
W004	0.00	1
W009	0.00	1
W008	0.00	1
W001	0.00	1
W006	0.00	1
W010	0.00	1
W007	0.00	1

Warehouses were ranked based on on-time delivery percentage using window functions, without joining additional tables since Warehouse_ID was available in the Orders table.

Delivery Agent Performance

1. Rank agents (per route) by on-time delivery percentage



SQL File 6*

```
-- Task 5: Delivery Agent Performance
-- 5.1: Rank agents (per route) by on-time delivery percentage
SELECT o.Route_ID, d.Agent_ID,
ROUND((SUM(DATEDIFF(o.Actual_Delivery_Date, o.Order_Date) = 0) * 100.0) / COUNT(*), 2)
AS On_Time_Percentage,
DENSE_RANK() OVER (PARTITION BY o.Route_ID ORDER BY
(SUM(DATEDIFF(o.Actual_Delivery_Date, o.Order_Date) = 0) * 1.0) / COUNT(*) DESC)
AS Agent_Rank FROM Orders o JOIN Delivery_Agents d ON o.Route_ID = d.Route_ID
WHERE o.Actual_Delivery_Date >= o.Order_Date GROUP BY o.Route_ID, d.Agent_ID;
```

Result 25 x

Route_ID	Agent_ID	On_Time_Percentage	Agent_Rank
R018	A019	0.00	1
R018	A017	0.00	1
R018	A008	0.00	1
R019	A010	0.00	1
R019	A004	0.00	1
R020	A036	0.00	1
R020	A046	0.00	1

Output

#	Time	Action	Message	Duration / Fetch
43	13:19:06	SELECT o.Route_ID, d.Agent_ID, ROUND(SUM(DATEDIFF(o.Actual_Delivery_Date, o.O...	Error Code: 1146: Table 'amazon.deliveryagents' doesn't exist	0.110 sec
44	13:21:10	SELECT o.Route_ID, d.Agent_ID, ROUND(SUM(DATEDIFF(o.Actual_Delivery_Date, o.O...	50 row(s) returned	0.031 sec / 0.000 sec
45	13:21:53	SELECT o.Warehouse_ID, ROUND(SUM(DATEDIFF(o.Actual_Delivery_Date, o.Order_Dat...	10 row(s) returned	0.000 sec / 0.000 sec
46	13:22:29	SELECT o.Route_ID, d.Agent_ID, ROUND(SUM(DATEDIFF(o.Actual_Delivery_Date, o.O...	50 row(s) returned	0.015 sec / 0.000 sec

Agent performance was evaluated by linking agents to routes and calculating on-time delivery percentage per route.

2. Find agents with on-time % < 80%.



SQL File 6*

```
-- 5.2: Find agents with on-time % < 80%.
SELECT d.Agent_ID,
ROUND((SUM(DATEDIFF(o.Actual_Delivery_Date, o.Order_Date) = 0) * 100.0) / COUNT(*), 2) AS On_Time_Percentage
FROM Orders o JOIN Delivery_Agents d ON o.Route_ID = d.Route_ID
WHERE o.Actual_Delivery_Date >= o.Order_Date GROUP BY d.Agent_ID HAVING On_Time_Percentage < 80;
```

Result 26 x

Agent_ID	On_Time_Percentage
A035	0.00
A029	0.00
A026	0.00
A048	0.00
A016	0.00
A009	0.00
A002	0.00

Output

#	Time	Action	Message	Duration / Fetch
44	13:21:10	SELECT o.Route_ID, d.Agent_ID, ROUND(SUM(DATEDIFF(o.Actual_Delivery_Date, o.O...	50 row(s) returned	0.031 sec / 0.000 sec
45	13:21:53	SELECT o.Warehouse_ID, ROUND(SUM(DATEDIFF(o.Actual_Delivery_Date, o.Order_Dat...	10 row(s) returned	0.000 sec / 0.000 sec
46	13:22:29	SELECT o.Route_ID, d.Agent_ID, ROUND(SUM(DATEDIFF(o.Actual_Delivery_Date, o.O...	50 row(s) returned	0.015 sec / 0.000 sec
47	13:54:01	SELECT d.Agent_ID, ROUND(SUM(DATEDIFF(o.Actual_Delivery_Date, o.Order_Date) = 0...	50 row(s) returned	0.000 sec / 0.000 sec

Task 5:

3. Compare average speed of top 5 vs bottom 5 agents using subqueries.



The screenshot shows the MySQL Workbench interface. The SQL editor contains a query that compares the average speed of the top 5 and bottom 5 agents. The query is as follows:

```
-- 5.3: Compare average speed of top 5 vs bottom 5 agents using subqueries.
133 • SELECT 'Top 5 Agents' AS Category,AVG(Avg_Speed) AS Avg_Speed
134 FROM (SELECT d.Agent_ID,AVG(r.Distance_KM / r.Average_Travel_Time_Min) AS Avg_Speed
135 FROM Orders o JOIN Delivery_Agents d ON o.Route_ID = d.Route_ID
136 JOIN Routes r ON o.Route_ID = r.Route_ID GROUP BY d.Agent_ID ORDER BY Avg_Speed DESC LIMIT 5) AS Top_Agents
137 UNION
138 SELECT 'Bottom 5 Agents' AS Category,AVG(Avg_Speed) AS Avg_Speed
139 FROM (SELECT d.Agent_ID,AVG(r.Distance_KM / r.Average_Travel_Time_Min) AS Avg_Speed
140 FROM Orders o JOIN Delivery_Agents d ON o.Route_ID = d.Route_ID JOIN Routes r ON o.Route_ID = r.Route_ID
141 GROUP BY d.Agent_ID ORDER BY Avg_Speed LIMIT 5) AS Bottom_Agents;
142
143 • SELECT d.Agent_ID,AVG(r.Distance_KM / r.Average_Travel_Time_Min) AS Avg_Speed
144 FROM Orders o JOIN Delivery_Agents d ON o.Route_ID = d.Route_ID
145 JOIN Routes r ON o.Route_ID = r.Route_ID GROUP BY d.Agent_ID ORDER BY Avg_Speed DESC LIMIT 5;
146
147 • SELECT d.Agent_ID,AVG(r.Distance_KM / r.Average_Travel_Time_Min) AS Avg_Speed
148 FROM Orders o JOIN Delivery_Agents d ON o.Route_ID = d.Route_ID
149 JOIN Routes r ON o.Route_ID = r.Route_ID GROUP BY d.Agent_ID ORDER BY Avg_Speed LIMIT 5;
```

The output window shows the results of the query:

Category	Avg_Speed
Top 5 Agents	2.8502637843697225
Bottom 5 Agents	0.15309500296267037



Category	Avg_Speed
Top 5 Agents	2.8502637843697225
Bottom 5 Agents	0.15309500296267037



Agent_ID	Avg_Speed
A004	3.76938775510204
A010	3.76938775510204
A030	2.278181818181818
A003	2.278181818181818
A050	2.1561797752808975

Agent_ID	Avg_Speed
A001	0.15120481927710847
A005	0.15120481927710847
A039	0.15120481927710847
A043	0.15120481927710847
A020	0.16065573770491803

Insight

Top-performing agents are significantly faster than bottom-performing agents, indicating a large performance gap.

◆ Recommendation

- Assign better routes to low-performing agents
- Provide training or monitoring
- Optimize slow routes

The large variation in speed suggests that route conditions or agent efficiency differ significantly.

Shipment Tracking Analytics

1.For each order, list the last checkpoint and time.

2.Find the most common delay reasons (excluding None)

Task 6:

The screenshot shows the MySQL Workbench interface with a query executed. The query is as follows:

```
150
151 -- Task 6: Shipment Tracking Analytics
152 -- 6.1: For each order, list the last checkpoint and time.
153 • SELECT Order_ID,MAX(Checkpoint_Time) AS Last_Checkpoint_Time
154 FROM Shipment_Tracking GROUP BY Order_ID;
155
```

The result grid displays the following data:

Order_ID	Last_Checkpoint_Time
O0001	2025-06-09
O0002	2025-06-23
O0003	2025-06-05
O0004	2025-06-29
O0005	2025-06-04
O0006	2025-07-18

The bottom panel shows the action output with the following log:

#	Time	Action	Message	Duration / Fetch
52	14:29:49	SELECT d.Agent_ID,AVG(r.Distance_KM / r.Average_Travel_Time_Min) AS Avg_Speed FR...	5 row(s) returned	0.000 sec / 0.000 sec
53	15:05:26	SELECT Order_ID,MAX(Checkpoint_Time) AS Last_Checkpoint_Time FROM Shipment_Tra...	300 row(s) returned	0.032 sec / 0.000 sec
54	15:06:16	SELECT * ,MAX(Checkpoint_Time) AS Last_Checkpoint_Time FROM Shipment_Tracking G...	Error Code: 1055. Expression #1 of SELECT list is not in GROUP BY clause and contains no...	0.015 sec
55	15:06:54	SELECT Order_ID,MAX(Checkpoint_Time) AS Last_Checkpoint_Time FROM Shipment_Tra...	300 row(s) returned	0.015 sec / 0.000 sec

The latest checkpoint for each shipment was identified using the maximum checkpoint timestamp.

The screenshot shows the MySQL Workbench interface with a query executed. The query is as follows:

```
155
156 -- 6.2:Find the most common delay reasons (excluding None).
157 • SELECT Delay_Reason,COUNT(*) AS Frequency
158 FROM Shipment_Tracking
159 WHERE Delay_Reason IS NOT NULL AND Delay_Reason <> 'None'
160 GROUP BY Delay_Reason ORDER BY Frequency DESC;
```

The result grid displays the following data:

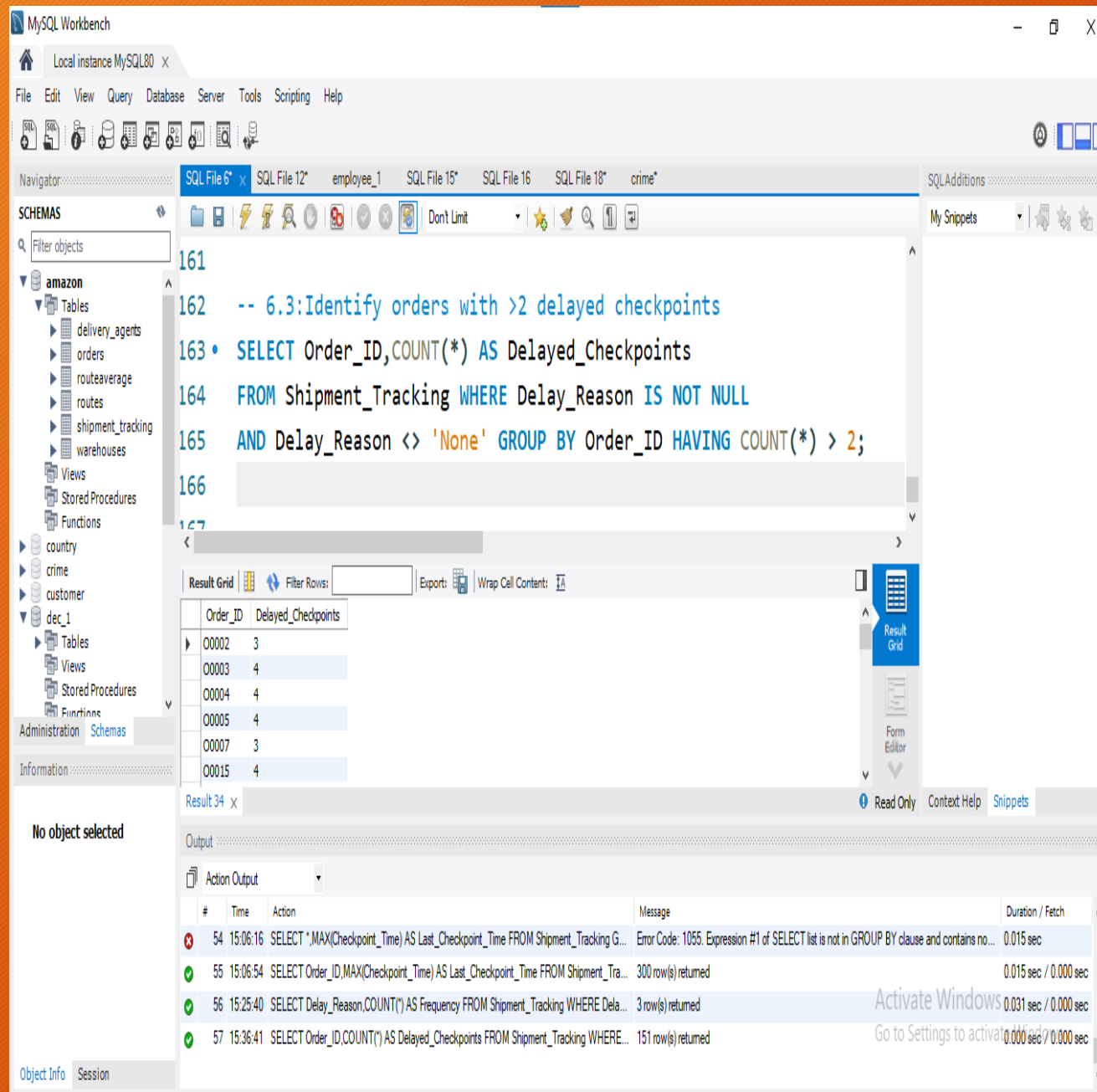
Delay_Reason	Frequency
Sorting Delay	272
Weather	263
Traffic	247

The bottom panel shows the action output with the following log:

#	Time	Action	Message	Duration / Fetch
53	15:05:26	SELECT Order_ID,MAX(Checkpoint_Time) AS Last_Checkpoint_Time FROM Shipment_Tra...	300 row(s) returned	0.032 sec / 0.000 sec
54	15:06:16	SELECT * ,MAX(Checkpoint_Time) AS Last_Checkpoint_Time FROM Shipment_Tracking G...	Error Code: 1055. Expression #1 of SELECT list is not in GROUP BY clause and contains no...	0.015 sec
55	15:06:54	SELECT Order_ID,MAX(Checkpoint_Time) AS Last_Checkpoint_Time FROM Shipment_Tra...	300 row(s) returned	0.015 sec / 0.000 sec
56	15:25:40	SELECT Delay_Reason,COUNT(*) AS Frequency FROM Shipment_Tracking WHERE Dela...	3 row(s) returned	0.031 sec / 0.000 sec

Null and non-delay values were excluded to accurately identify the most common delay reasons.

3. Identify orders with >2 delayed checkpoints



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
161
162 -- 6.3:Identify orders with >2 delayed checkpoints
163 • SELECT Order_ID,COUNT(*) AS Delayed_Checkpoints
164 FROM Shipment_Tracking WHERE Delay_Reason IS NOT NULL
165 AND Delay_Reason <> 'None' GROUP BY Order_ID HAVING COUNT(*) > 2;
166
```

The query results are displayed in a table with the following data:

Order_ID	Delayed_Checkpoints
O0002	3
O0003	4
O0004	4
O0005	4
O0007	3
O0015	4

The bottom panel shows the Action Output with the following log entries:

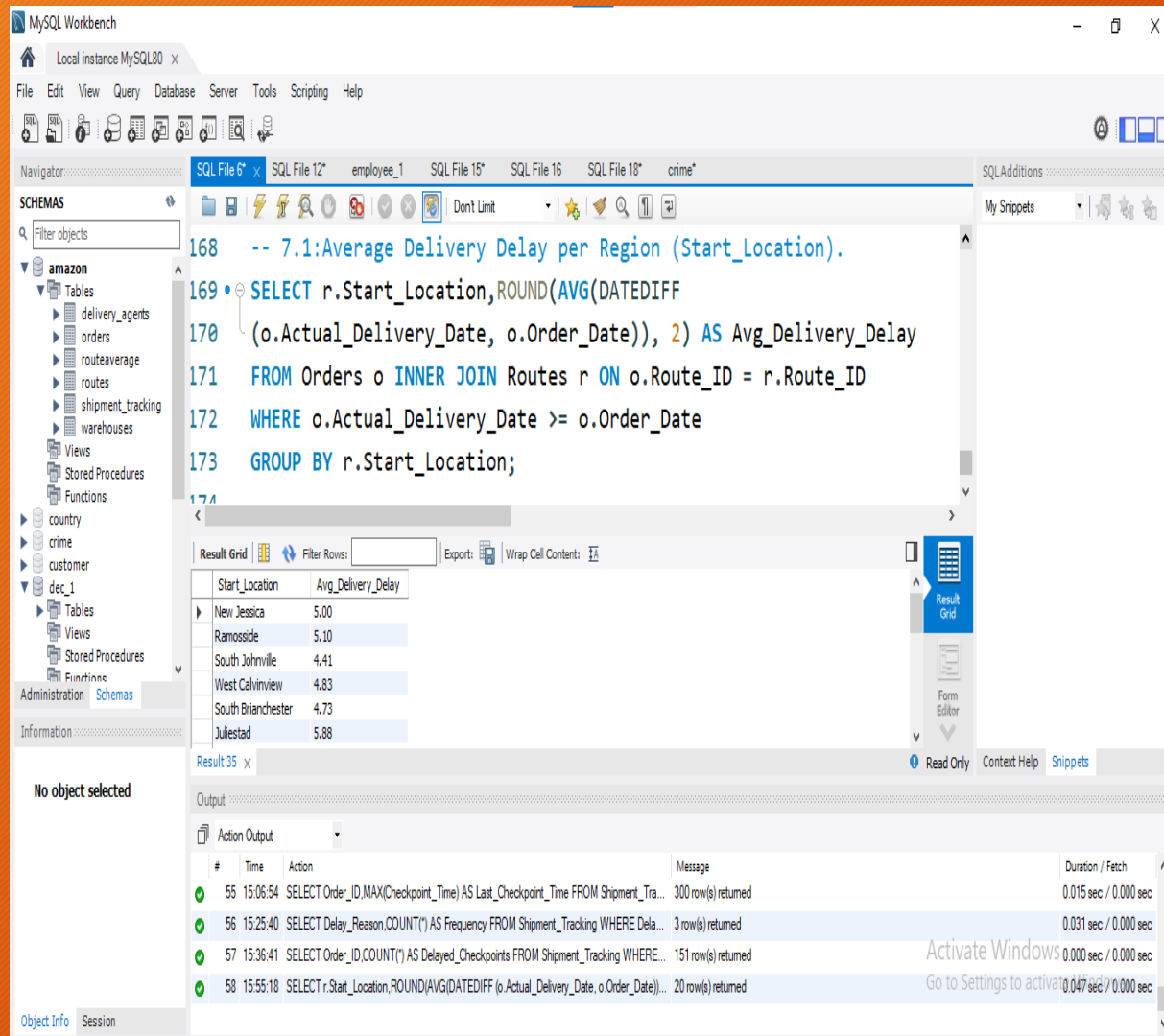
#	Time	Action	Message	Duration / Fetch
54	15:06:16	SELECT *,MAX(Checkpoint_Time) AS Last_Checkpoint_Time FROM Shipment_Tracking G...	Error Code: 1055. Expression #1 of SELECT list is not in GROUP BY clause and contains no...	0.015 sec
55	15:06:54	SELECT Order_ID,MAX(Checkpoint_Time) AS Last_Checkpoint_Time FROM Shipment_Tra...	300 row(s) returned	0.015 sec / 0.000 sec
56	15:25:40	SELECT Delay_Reason,COUNT(*) AS Frequency FROM Shipment_Tracking WHERE Dela...	3 row(s) returned	0.031 sec / 0.000 sec
57	15:36:41	SELECT Order_ID,COUNT(*) AS Delayed_Checkpoints FROM Shipment_Tracking WHERE...	151 row(s) returned	0.000 sec / 0.000 sec

Orders with more than two delayed checkpoints were identified to highlight shipments with repeated disruptions.

- Identified orders with **more than 2 delayed checkpoints**
- Delay defined as checkpoints with a valid **Delay_Reason** (excluding 'None')
- Counted delayed checkpoints for each order
- Highlighted shipments with **repeated disruptions**

Advanced KPI Reporting

1. Average Delivery Delay per Region (Start_Location).



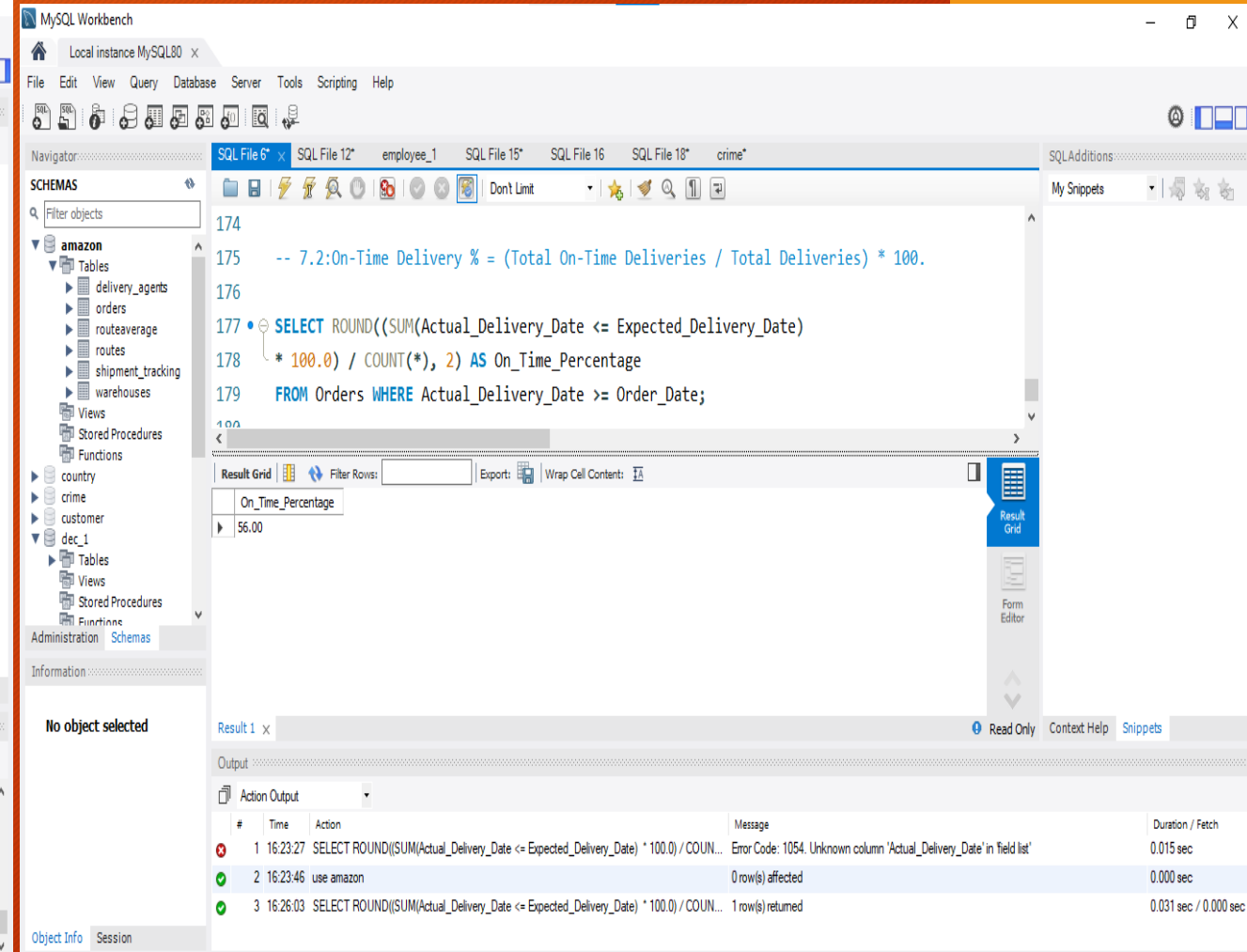
The screenshot shows the MySQL Workbench interface with a SQL query in the editor and its results in the Result Grid. The query calculates the average delivery delay per region by joining the Orders and Routes tables and using the DATEDIFF function to find the difference between the actual and expected delivery dates.

```
168 -- 7.1: Average Delivery Delay per Region (Start_Location).
169 SELECT r.Start_Location, ROUND(AVG(DATEDIFF
170 (o.Actual_Delivery_Date, o.Order_Date)), 2) AS Avg_Delivery_Delay
171 FROM Orders o INNER JOIN Routes r ON o.Route_ID = r.Route_ID
172 WHERE o.Actual_Delivery_Date >= o.Order_Date
173 GROUP BY r.Start_Location;
```

Start_Location	Avg_Delivery_Delay
New Jessica	5.00
Ramosside	5.10
South Johnville	4.41
West Calvinview	4.83
South Brianchester	4.73
Juliestad	5.88

Average delivery delay was calculated per region by joining Orders with Routes to access start location data.

2. On-Time Delivery % = (Total On-Time Deliveries / Total Deliveries) * 100.



The screenshot shows the MySQL Workbench interface with a SQL query in the editor and its results in the Result Grid. The query calculates the on-time delivery percentage by counting the number of orders where the actual delivery date is less than or equal to the expected delivery date, divided by the total number of orders.

```
174
175 -- 7.2: On-Time Delivery % = (Total On-Time Deliveries / Total Deliveries) * 100.
176
177 SELECT ROUND((SUM(Actual_Delivery_Date <= Expected_Delivery_Date)
178 * 100.0) / COUNT(*), 2) AS On_Time_Percentage
179 FROM Orders WHERE Actual_Delivery_Date >= Order_Date;
```

On_Time_Percentage
56.00

On-time delivery was defined as deliveries completed on or before the expected delivery date.

Task 7:

3.Average Traffic Delay per Route.

Average traffic delay per route was calculated to analyze route efficiency and identify congestion-prone routes.

Final Dashboard Query(combined query)

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SQL File 6* x SQL File 12* employee_1 SQL File 15* SQL File 16 SQL File 18* crime*

SQL Additions

SCHMAS

Filter objects

amazon

Tables

delivery_agents orders routeaverage routes shipment_tracking warehouses

Views Stored Procedures Functions

country crime customer dec_1

Tables Views Stored Procedures Functions

Administration Schemas

Information

182

183 -- 7.3:Average Traffic Delay per Route.

184 • SELECT Route_ID,Start_Location,End_Location,

185 ROUND(AVG(Traffic_Delay_Min), 2) AS Avg_Traffic_Delay_Min

186 FROM Routes GROUP BY Route_ID, Start_Location, End_Location;

187

Result Grid

Filter Rows

Exports

Wrap Cell Contents

Result Grid

Form Editor

Route_ID	Start_Location	End_Location	Avg_Traffic_Delay_Min
R001	New Aimeechester	Sabrinaland	36.00
R002	South Brianchester	Huffmanmouth	22.00
R003	East Amy	South Katherinemouth	11.00
R004	Ramosside	Stevensview	3.00
R005	New Jessica	Davismouth	26.00
R006	Lake Robinhaven	East Christopherville	42.00
R007	Aaronhaven	Jeanneberg	51.00
R008	South Davidborough	Carsonberg	34.00
R009	Belfurt	Lake Marvin	46.00

Read Only Context Help Snippets

Result 3 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
3	16:26:03	SELECT ROUND(SUM(Actual_Delivery_Date <= Expected_Delivery_Date) * 100.0) / COU...	1 row(s) returned	0.031 sec / 0.000 sec
4	16:28:45	SELECT DATEDIFF(Actual_Delivery_Date, Order_Date) AS diff, COUNT(*) FROM Ord...	8 row(s) returned	0.016 sec / 0.000 sec
5	16:40:54	SELECT Route_ID,Start_Location,End_Location,ROUND(AVG(Traffic_Delay_Min),2) AS A...	20 row(s) returned	0.031 sec / 0.000 sec

Object Info Session

```
187
188 • SELECT r.Route_ID,r.Start_Location,
189 -- KPI 1: Avg Delivery Delay
190 ROUND(AVG(DATEDIFF(o.Actual_Delivery_Date, o.Order_Date)), 2) AS Avg_Delivery_Delay_Days,
191 -- KPI 2: On-Time Delivery %
192 ROUND((SUM(o.Actual_Delivery_Date <= o.Expected_Delivery_Date) * 100.0) / COUNT(*), 2)AS On_Time_Delivery_Percentage,
193 -- KPI 3: Avg Traffic Delay
194 ROUND(AVG(r.Traffic_Delay_Min), 2) AS Avg_Traffic_Delay_Min FROM Orders o INNER JOIN Routes r ON o.Route_ID =
195 r.Route_ID WHERE o.Actual_Delivery_Date >= o.Order_Date GROUP BY r.Route_ID, r.Start_Location;
```

output

	Route_ID	Start_Location	Avg_Delivery_Delay_Days	On_Time_Delivery_Percentage	Avg_Traffic_Delay_Min
▶	R005	New Jessica	5.00	53.33	26.00
	R004	Ramosside	5.10	50.00	3.00
	R013	South Johnville	4.41	47.06	38.00
	R012	West Calvinview	4.83	33.33	29.00
	R002	South Brianchester	4.73	68.18	22.00
	R020	Juliestad	5.88	29.41	23.00
	R008	South Davidborough	5.44	68.75	34.00
	R011	Lowemouth	5.55	55.00	25.00

Delivery Delay Days

5.29

Average of...

On Time Delivery

56.05

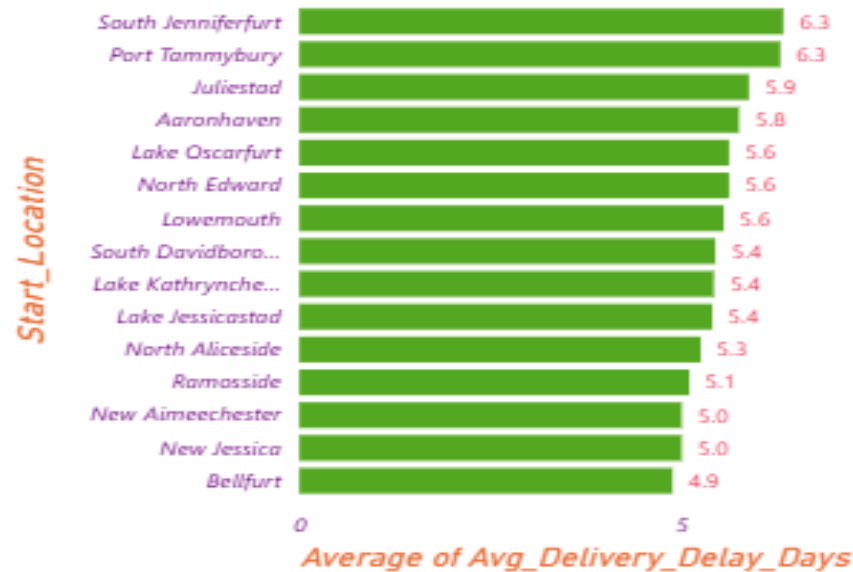
Average of...

Avg Traffic Delay

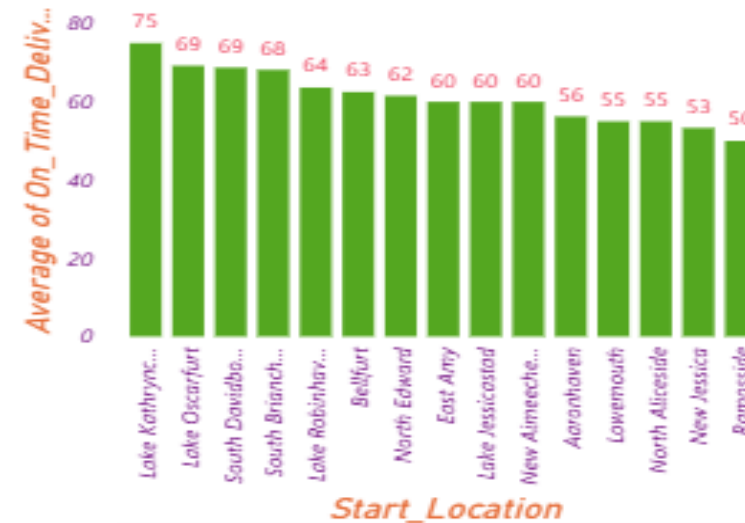
28.85

Average of...

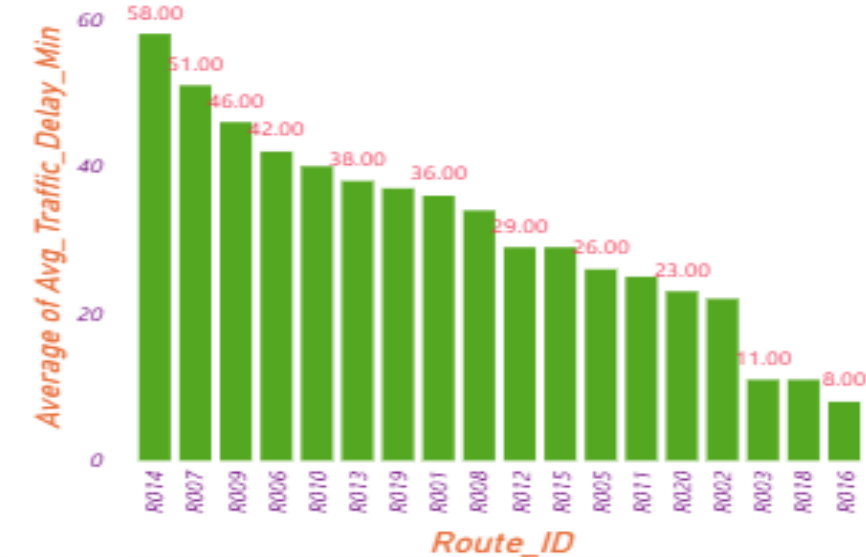
Average of Avg_Delivery_Delay_Days by Start_Location



Average of On_Time_Delivery_Percentage by Start_Location



Average of Avg_Traffic_Delay_Min by Route_ID



KPIs were visualized using Power BI dashboards to provide interactive insights into delivery performance, traffic delays, and operational efficiency.

Insights :

- On-time delivery rate is relatively low (56%)
- Average delivery delay is over 5 days
- Traffic congestion significantly affects delivery performance(28.85)

Recommendations:

- Optimize high-delay routes
- Improve delivery planning and scheduling
- Focus on underperforming regions
- Reduce traffic impact using alternative routes
- Use data-driven decisions for logistics improvement

Conclusion:

The analysis highlights key inefficiencies in delivery performance, with traffic delay being a major contributing factor.

“This project demonstrates how data analytics can be used to improve operational efficiency and decision-making in logistics.”

Video Link



https://drive.google.com/drive/folders/1jmt5QXrMv8KZZeGleKHnzmkD05UDxqZt?usp=drive_link